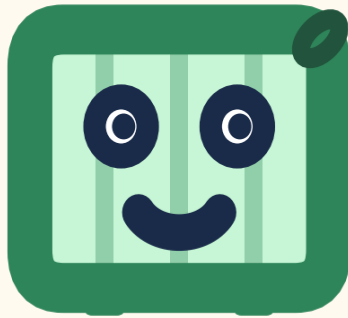


SPRING

How IoC, DI, and AOP actually work — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Inversion of Control
2. The Bean Lifecycle
3. @Transactional Is a Proxy
4. The Self-Invocation Pitfall
5. Autowiring: Type, Then Name
6. Spring MVC: The Request Journey
7. Cheat Sheet

Spring's magic is really just a small set of consistent ideas -- dependency injection, proxies, and a bean lifecycle -- applied everywhere. Once these click, the rest of the framework stops feeling like magic.

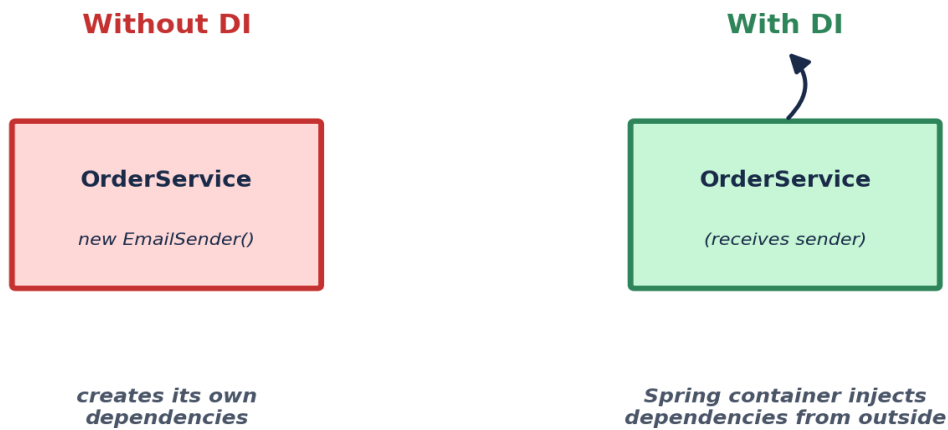
CHAPTER 1

Inversion of Control

Without dependency injection, a class constructs its own dependencies directly -- `new EmailSender()` right there in the constructor. That hardcodes exactly which implementation is used, making the class harder to test (you can't substitute a mock) and harder to reconfigure.

With DI, a class simply declares what it needs, and the Spring container supplies a concrete implementation from outside -- via constructor injection. The class never calls `new` on its own dependencies at all.

Inversion of Control



IN SHORT: A class shouldn't construct its own dependencies. Declare what you need; let the container hand it to you.

CHAPTER 2

The Bean Lifecycle

A bean doesn't simply spring into existence fully formed. Spring instantiates it, injects its dependencies, then invokes any `@PostConstruct` method -- which is the right place for initialization logic that depends on those now-injected values, since it's guaranteed to run only after they're all set.

On container shutdown, the same bean's `@PreDestroy` method runs, giving it a chance to release resources (close connections, flush buffers) before the container discards it.

The Bean Lifecycle



IN SHORT: Construction and dependency injection happen before `@PostConstruct` ever runs. Never assume injected fields are ready inside the constructor itself.

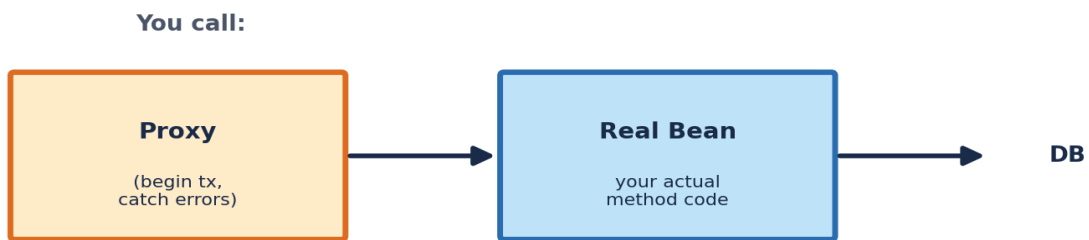
CHAPTER 3

@Transactional Is a Proxy

Spring implements @Transactional (and @Async) using dynamic proxies. The bean you actually inject and call isn't your raw class -- it's a proxy wrapping it.

Calling a transactional method actually calls the proxy first. The proxy begins a transaction, invokes your real method, and commits or rolls back based on the outcome -- all of this happens transparently, without your business logic needing to know anything about transaction management.

@Transactional Is a Proxy



The bean you inject is often actually a dynamic proxy wrapping the real object -- that's how @Transactional and @Async add behavior transparently.

IN SHORT: The object Spring injects is often not your class directly, but a proxy wrapping it -- which is exactly why bypassing the proxy breaks the magic (see next chapter).

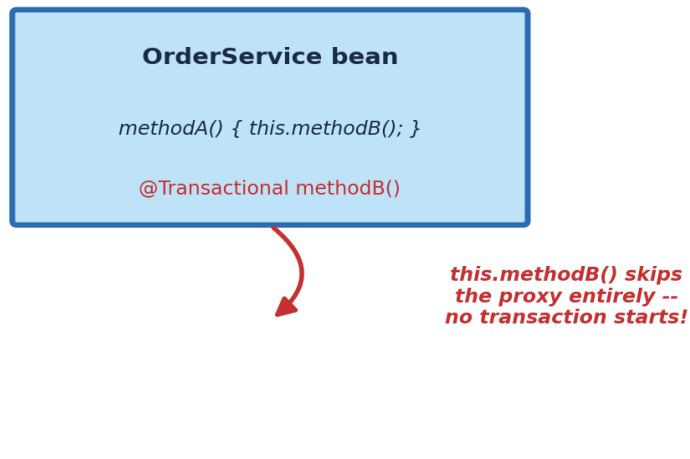
CHAPTER 4

The Self-Invocation Pitfall

Since `@Transactional` only works through the proxy, calling a transactional method via `this.methodB()` from within the same class skips the proxy entirely -- the call goes directly to the real object, and no transaction ever starts.

This is one of the most common, genuinely confusing Spring bugs: the code compiles fine, runs without error, and simply silently doesn't do what the annotation promised.

The Self-Invocation Pitfall



IN SHORT: `@Transactional` (and `@Async`, and `@Cacheable`) only work when called from OUTSIDE the bean. A same-class `this.method()` call bypasses the proxy silently.

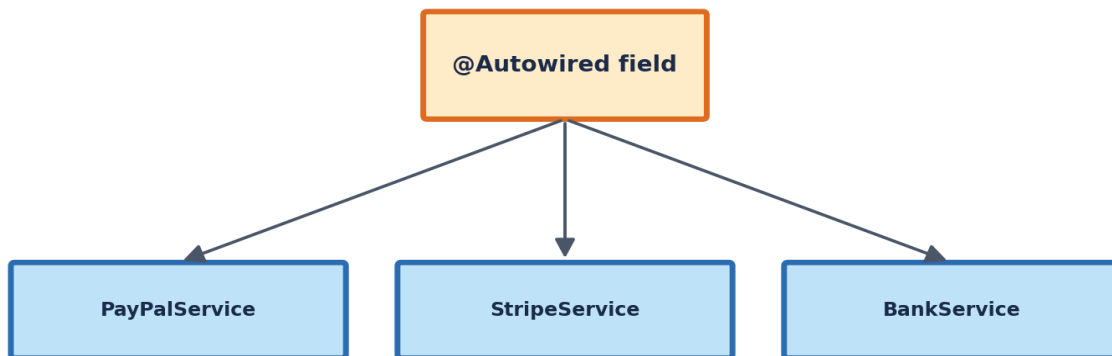
CHAPTER 5

Autowiring: Type, Then Name

@Autowired first tries to resolve a dependency by type. If exactly one matching bean exists, done. If multiple beans implement the same interface, Spring falls back to matching the field or parameter name against bean names.

If that still doesn't resolve to a single candidate, Spring throws `NoUniqueBeanDefinitionException` at startup -- resolved explicitly with `@Qualifier("beanName")` to pick a specific one, or `@Primary` to designate a sensible default among the candidates.

Autowiring: Type, Then Name



*Multiple beans implement PaymentService -> ambiguous!
Use @Qualifier("stripeService") or @Primary to resolve it.*

IN SHORT: Multiple beans of the same interface type is ambiguous by default. @Qualifier or @Primary makes the choice explicit rather than accidental.

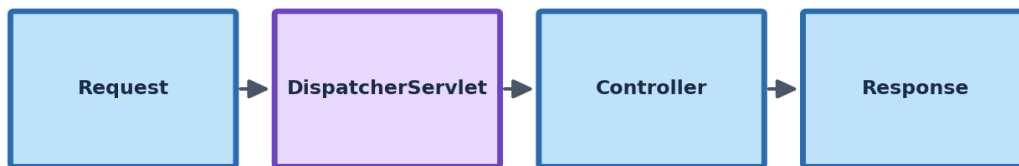
CHAPTER 6

Spring MVC: The Request Journey

Every incoming HTTP request to a Spring MVC application passes through a single front controller: the `DispatcherServlet`. It doesn't handle the request's business logic itself -- it looks up which controller method matches the request's path and method, and delegates to it.

The controller method runs, returns a result, and the `DispatcherServlet` (via an appropriate `HttpMessageConverter`, typically Jackson for JSON) serializes that result into the actual HTTP response sent back to the client.

Spring MVC: The Request Journey



Every request funnels through one front controller (`DispatcherServlet`), which routes to the matching `@RequestMapping` method.



IN SHORT: One front controller routes every request. Your `@RequestMapping` methods handle the specific logic; the `DispatcherServlet` handles getting the request there and the response back.

CHAPTER 7

Cheat Sheet

The annotations and concepts you'll use daily.

SPRING CHEAT SHEET



Core

@Component/@Service/@Repository

@Autowired - inject by type

@Qualifier - resolve ambiguity

@Configuration + @Bean

Scopes

singleton (default) - one per container

prototype - new instance each time

request/session - web-scoped

AOP & Transactions

@Transactional - proxy-based

@Async - runs on another thread

Self-invocation bypasses the proxy!

Web

@RestController = @Controller + @ResponseBody

@RequestMapping/@GetMapping/@PostMapping

@RequestBody / @PathVariable / @RequestParam

Testing

@SpringBootTest - full context

@WebMvcTest - web layer only

@DataJpaTest - JPA layer only

@MockBean - mock + register in context